

REDRED

The Same Problem, Solved Twice

Two completely different detectors — YOLOv7 and RF-DETR+SAM2 — each independently reached a perfect score. This deck is the record of what stayed the same between the two runs, and what didn't.

YOLOv7

Count / Order / Time F1 100%

RF-DETR + SAM2

Count / Order / Time F1 100%

Bottom line

Two architectures, one identical result

What this deck covers, in the order it happened

Pipeline improvement → a different model (YOLO11) fails → root cause identified → literature and industry research → RF-DETR adopted → porting fails → diagnosis → data expanded with SAM2 → redesign → 100% reproduced → comparison.

00 Problem definition & evaluation criteria + background (3-way scoring, leaderboard)

01 YOLOv7 — self fine-tuning fails → pivot to pipeline engineering

02 YOLOv7 — the full 4-layer pipeline, in detail → 100%

03 YOLO11 attempt → "mAP is higher, so why doesn't it show up on video?"

04 Literature & industry research → recommended techniques fail → RF-DETR adopted

05 Adopting the full RF-DETR stack → porting alone isn't enough

06 Class-specific error diagnosis → raw data is the root fix

07 SAM2 expands raw data → probabilistic redesign → per-class config system → 100%

08 RF-DETR's confidence-calibration advantage → less hardcoding + RTF

09 Comparing the two architectures

10 What this competition taught us

11 Conclusion

The task: 5 cameras, 60 products, one CSV to get right

Five cameras film the same shelf from five different angles. The task: recognize purchase and return events across 60 product classes, and output the resulting inventory and payment data as a CSV.



EVALUATION ITEM

POINTS

Inventory recognition accuracy (Count / Order / Time F1)

40

RTF — processing speed (RTF ≤ 1 earns full marks)

20

Qualitative evaluation + presentation

40

Five diagnostic tools we built before touching the model

The competition never published an official scoring formula. Every diagnostic and automation tool below was built from scratch — and every one of them gets reused across both architectures later in this deck.

- ✓ **3-way scoring methodology (count / order / time, `tools/score_methods.py`)**
Count alone can look "correct" even when the order is wrong, and a single misordered pair can wrongly fail unrelated classes — the three metrics cover each other's blind spots.
- ✓ **LCS backtracking algorithm**
When Order F1 shows a mismatch, this pinpoints exactly which pair of events swapped places — down to the index.
- ✓ **Experiment-log leaderboard (`output/leaderboard.html`)**
Every one of 55+ runs is logged with what motivated it, what problem it exposed, and what came next — an expandable lab notebook, not just a scoreboard.
- ✓ **`debug_frame_counts.csv` replay tool**
Replays raw per-frame counts locally, with no server needed, to separate "the model genuinely missed it" from "the pipeline logic is buggy."
- ✓ **Candidate-time back-calculation (`fire_time` formula)**
Works backward from an event's confirmed firing time to tune `confirm_frames` precisely — reused verbatim in both the YOLOv7 and RF-DETR sessions.

Self fine-tuning fails → pivot to pipeline engineering

Starting point: the competition-provided weights (`yolov7_custom.pt` , mAP@0.5 98.1%, trained on roughly 200,000 images).

ATTEMPT

RESULT

VERDICT

Cut&Paste augmentation (5,000 images) + self fine-tune (30 epochs)

mAP@0.5 0.9904 → 0.9840 (dropped)

Not adopted

Why we pivoted — the lighting, shadow, and object-placement distribution of the augmented images diverged from the real test footage, so the model drifted toward the synthetic distribution instead of improving.

Conclusion — touching the model itself is risky. Keep the provided weights untouched, and improve the pipeline that sits on top of them instead.

The YOLOv7 pipeline: four layers on top of a frozen model

Because the detector weights are frozen, every gain from here on comes from four layers of pipeline logic sitting above them — each layer solving a problem the previous one couldn't.

LAYER 1

Multi-Camera Fusion

Weighted median + per-class
quorum override + camera
whitelist

LAYER 2

Occlusion-Aware Weighting

Global → per-class → per-camera
exclusion, refined across three
stages

LAYER 3

SORT Tracker

`max_age=15` — bridges short
occlusion gaps

LAYER 4

Hysteresis State Machine

STABLE/CANDIDATE states +
per-class, per-direction confirm
timing

Why four layers — each layer targets a failure mode the previous one leaves behind: physical blind spots (L1), transient hand occlusion (L2), short tracking gaps (L3), and precise event-timing disputes between classes (L4). The next three slides go through each in turn.

Layer 1 — Multi-Camera Fusion

Quorum override + camera whitelist — the default rule is a weighted median across all 5 cameras (majority agreement). Two class-specific exceptions sit on top of that default.

EXCEPTION 1

CLASS_QUORUM_OVERRIDE

Some products are physically visible from only 1-2 of the 5 cameras because of shelf placement — a structural blind spot that no "majority rule" can satisfy. For those classes (8 in total), the required agreement count is relaxed to 1-2 cameras.

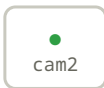
EXCEPTION 2

CLASS_CAM_WHITELIST

Per-camera logs showed that specific cameras (e.g. one angle repeatedly confusing `campbells` with `milano` or `dove_white`) were the source of recurring misclassification — those cameras are excluded outright for the affected classes.

Layer 2 — Occlusion-Aware Camera Weighting

Telling "briefly blocked" apart from "actually changed" — global average → per-class judgment → individual camera exclusion, refined across three stages.



cam0 reads 0 while $N = 2$ or more of the remaining four read positive → cam0 is treated as occluded and excluded from the vote. The check uses only that class's own confidence — never a global average.

Occlusion detection rate

8% → 27–38%

False-positive events

reduced to 0

Layers 3–4 — Tracker + State Machine → 100%

③ SORT tracker

`max_age=15` — bridges measured occlusion gaps of 0.4–1.5 s. Nearly free in RTF terms: it adds only CPU-side Kalman/loU matching, with no extra GPU inference.

④ Hysteresis state machine

A two-state STABLE/CANDIDATE machine with `WINDOW_SIZE=25` and `CONFIRM_FRAMES=30`, plus per-class and per-direction `per_class_confirm` tuning.

STABLE (committed)

— value change detected →

CANDIDATE

— held for CONFIRM_FRAMES →

STABLE (event confirmed)

Count F1

100%

Order F1

100%

Time F1

100%

RTF

0.763

YOLO11: higher mAP, worse real-world accuracy

Hypothesis: a newer, lighter architecture (YOLO11) should beat the competition weights on both accuracy and speed.

ATTEMPT

RESULT

YOLO11m retrained (18,392 images, 50 epochs)

mAP 98.4% (comparable or higher) · Order F1 73.4% (−23.2 pp)

YOLO11m + Knowledge Distillation

Order F1 91.1% (−7.5 pp vs. YOLOv7)

The real cause wasn't the model architecture — it was raw data volume. The provided YOLOv7 weights were trained on roughly 200,000 images; the team's own retraining set had only 18,000 (a 10×+ gap). mAP on a standard benchmark can look similar even when the model has nowhere near enough coverage of the real footage's angles, lighting, and occlusion patterns.

This realization set the strategy going forward — "the next time we try a different model, the question isn't just architecture — it's how we secure enough raw data."

Six established techniques tested, all failed → RF-DETR adopted

Even after YOLOv7 reached 100%, one question remained: is this result specific to YOLOv7, or just luck? To find out, the team surveyed recent papers and industry practice — and tested every candidate.

TECHNIQUE	MEASURED RESULT	REASON FOR REJECTION
ByteTrack	count 93.6% / order 91.7% / time 89.9%	Keeping low-confidence detections either blocks new tracks or triggers over-firing
SeqNMS	Suppressed legitimate events as well	Filters out sparse-but-real signals along with the noise
EMA smoothing	Suppressed events	No reset mechanism — stale values persist
Ghost Detector	Not applicable	Requires retraining the source paper's own model — can't be reused here
WBF ensemble	False-positive explosion	Confidence-calibration mismatch between the two ensembled models
BEV 3D fusion	Not applicable	Camera calibration data was never provided by the competition

Conclusion — the ceiling on temporal-smoothing tricks was clear. Time to switch the detector architecture entirely: adopt **RF-DETR** (Transformer-based, NMS-free, and reported to have better-calibrated confidence than CNN detectors).

First attempt: reusing YOLOv7's code plateaus at 85.6%

Developed on a separate branch (`feat/rfdetr-sam2`) — `main` and the final submission were never touched. Adopting RF-DETR meant rebuilding the entire stack, from preprocessing to decision logic.

- 1 YOLOv7 code ported as-is**
First scored with `run_pipeline_rfdetr.py`, which reused `event_detector.py` / `multi_view_fusion.py` unchanged — 10-epoch Order F1 85.2%.
- 2 50-epoch retrain (no augmentation) → got worse**
Order F1 dropped to 82.6% — 9 new classes started over-firing. This confirmed, for the second time in the project, that higher mAP doesn't translate into better pipeline performance (the same lesson as YOLO11).
- 3 RTF retuning + accumulated fixes → Order F1 85.6%**
Adjusted `skip=3` / `conf=0.35`, added `CLASS_CONF_OVERRIDE` plus a per-class whitelist/quorum — following the same playbook as the YOLOv7 session.

Copying YOLOv7's diagnostics and tuning stack wasn't enough to reach 100% — it became clear that two things were needed at once: (1) more raw data, and (2) a pipeline redesigned around RF-DETR's own characteristics.

Diagnosis: the real problem was missing training data

Breaking the 85.6% plateau down class by class.

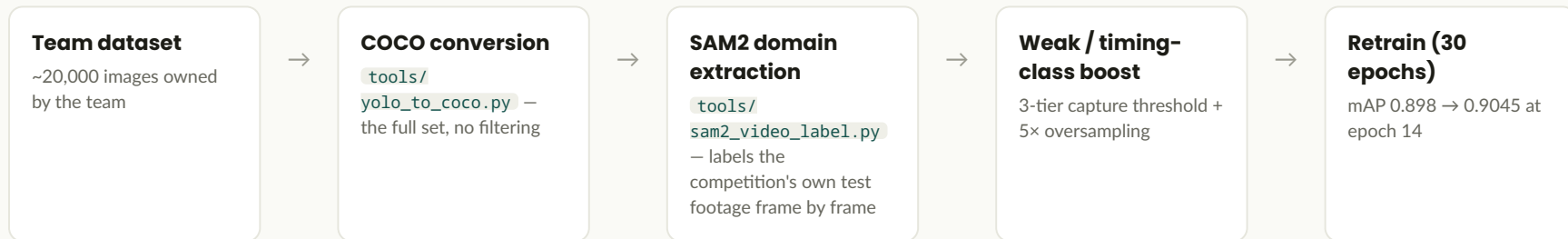
Symptom — specific classes (`campbells` , `hunts_sauce` , `cheerios` , and others) kept failing to be detected, in a structural way.

Diagnosis — the same pattern as YOLO11. Not a model-architecture problem, not a pipeline-logic problem — the training data simply didn't contain enough exposure to those specific classes.

Conclusion — rather than tune more parameters, expanding the raw data is the root-level fix. The competition only ever handed over **weights trained on 200,000 images** — never the original images themselves — so growing the dataset had to be done in-house.

SAM2: extracting training data straight from the test video

The competition handed over trained weights, not the 200,000 images behind them. So instead of chasing that number, the team pointed SAM2 at the one thing it had unlimited access to: the competition's own test footage.



- 1 All ~20,000 team images converted to COCO format**
The full set, not a hand-picked subset — there was no "select the 200k-equivalent images" step, because the 200k images were never in the team's hands to begin with.
- 2 SAM2 turns the competition's own test video into training frames**
It extracts and labels additional domain data directly from the target footage — camera angle, lighting, and occlusion patterns match the actual scoring video by construction, closing the train/test distribution gap that no amount of generic augmentation could.
- 3 A 3-tier capture threshold, not a blanket lower bar**
Lowering SAM2's confidence threshold everywhere risks flooding good classes with noisy labels — so the threshold was lowered only for the classes that actually needed it, in two different degrees. Details on the next slide.

Three tiers of capture threshold + 5× oversampling

TIER	CLASSES	SAM2 CAPTURE THRESHOLD	REASONING
Weak (structurally undetected)	aunt_jemima, hunts_sauce, campbells, chewy_dips_chocolate_chip, cheerios	0.3 → 0.1	Lowered the most — these classes had essentially zero usable signal at the default threshold
Timing-issue (detected, mistimed)	bumblebee_albacore, hariibo, milano, chewy_dips_peanut_butter, lindt	0.3 → 0.2	Already detected reasonably well — a smaller cut avoids re-introducing noise into classes that aren't actually missing
Everything else	remaining 50 classes	0.3 (unchanged)	No evidence of a data problem — left alone

Amplification, not selection — images containing the two boosted tiers were then **oversampled 5×** in the training set (duplicated, not cherry-picked), so the model sees them far more often per epoch without touching the label quality of the rest of the dataset.

mAP — pass 1 (no augmentation) → pass 2 (SAM2-augmented, epoch 14)

0.898 → 0.9045

EMA mAP@0.5:0.95 — measured on a different basis than YOLOv7's @0.5 mAP, so the two are not directly comparable

Shallower than YOLOv7's four layers — and still 100%

Instead of porting YOLOv7's layers, `src/rfdetr_native_pipeline.py` was designed from scratch around RF-DETR's own properties. The next three slides go through each piece in depth.

01

Binary Presence

No count concept — ground-truth inventory is always 0/1

02

Noisy-OR Fusion

$P = 1 - \prod (1 - \text{conf}_i)$ replaces YOLOv7's four layers with one formula

03

Hysteresis + Adaptive Confirm

Confirmation time auto-interpolated from signal strength, per direction

04

Per-Class JSON Config

12 tunable dimensions, zero code changes — `relabel_regions` hardcoding removed

Count F1

100%

Order F1

100%

Time F1

100%

Structure

shallower

One formula standing in for four layers

The redesign starts from a fact the YOLOv7 session had already proven: across all 105 ground-truth events, the inventory of any single class is never anything but 0 or 1. Counting was never the real problem — presence was.

$$P = 1 - \prod (1 - \text{conf}_i)$$

Every active camera i contributes independent evidence that "this class is present." Combining the cameras as probabilities means **one confident camera can outweigh several uncertain ones** — the decision no longer depends on how many cameras agree, only on how convincing the evidence is in total.

YOLOV7 — QUORUM VOTING

needs N cameras $\geq$ threshold

A hard camera-count rule. Confidence values themselves are never trusted directly — they only gate whether a camera's vote counts at all, so the vote-counting logic (whitelist, quorum override, occlusion exclusion) has to do all the work.

RF-DETR — NOISY-OR FUSION

$P \geq 0.7$ combined probability

Two cameras independently reporting 0.5 confidence combine to $P = 1 - (0.5 \times 0.5) = 0.75$ — already past the 0.7 default threshold, without ever having to define "how many cameras count as a quorum" for that class.

Why this only works for RF-DETR — Noisy-OR treats each confidence score as a real probability, which is only valid if the detector's confidence is actually calibrated (a 0.7 really does mean "roughly 70% likely to be right"). YOLOv7's confidence couldn't be trusted at that level — which is exactly why it needed four layers of workaround instead of one formula. More on this in the confidence-calibration slide.

Every parameter per class — and zero code changes to retune

All class-specific behavior lives in one file, `config/rfdetr_native_class_config_v2_reinforced.json`, loaded into a `ClassConfig` object with 12 independently tunable dimensions. Where YOLOv7's overrides are Python dicts baked into `multi_view_fusion.py` — edit, commit, push, pull, rerun — retuning RF-DETR is a JSON edit and nothing else.

Camera selection

`whitelist` — restrict which cameras vote for a class
`quorum` / `conf` — per-class agreement count and effective confidence floor

Event timing

`confirm_frames` — either one number, or `{"0": purchase, "1": return}` when the two directions need opposite tuning
`refractory_frames`, `min/max_confirm_frames` +
`adaptive_confirm_classes` (opt-in signal-strength interpolation)

Fusion sensitivity

`presence_threshold` — per-class override of the default 0.7 Noisy-OR cutoff
`duplicate_penalty` — e.g. `milano: 1.0` disables the default 0.5× penalty for classes where duplicate boxes are a real detection, not noise

Signal shaping

`window_size` — sliding-window length for the presence vote
`low_thresh` — how easily a class is allowed to register as "gone"

Example — milano (class 42) — `confirm_frames: 45`, `duplicate_penalty: 1.0`. Milano genuinely produces duplicate boxes on legitimate detections, so the penalty that protects every other class from double-counting would only hurt this one — one line in the JSON turns it off, for this class alone.

Why RF-DETR could get away with a simpler pipeline

A model is "well-calibrated" if, among all predictions it labels with confidence 0.9, roughly 90% actually turn out correct — only then can that number be trusted as a probability.

	YOLOV7	RF-DETR
Confidence reliability	Can't be trusted at face value	Relatively well-calibrated
What that required	Quorum + whitelist + occlusion weighting + tracker (4 layers of workaround)	A single probability formula (Noisy-OR)
RTF	0.763	0.6856

RTF — despite the heavier transformer backbone, RF-DETR's end-to-end pipeline processes faster than YOLOv7's. Both clear the competition's RTF ≤ 1 threshold for full marks (20/20) regardless.

Two completely different architectures, one identical 100%

ITEM	YOLOV7	RF-DETR + SAM2
Detector weights	Competition-provided (200k images, pretrained)	Team's 20k images + SAM2 domain data, mAP 0.898 → 0.9045
First self-improvement attempt	Cut&Paste fine-tuning → mAP dropped	YOLOv7 code ported as-is → Order F1 stuck at 82.6–85.6%
Multi-view fusion	Weighted median + quorum + whitelist (4-layer stack)	Noisy-OR probability fusion (single formula)
Occlusion handling	Global → per-class → per-camera weighting + SORT tracker	Adaptive confirm (signal-strength interpolation)
Per-class tuning surface	Python dicts in <code>multi_view_fusion.py</code> — code change + redeploy	12-dimension JSON config — no code change
Data-scarcity response	n/a — used the provided weights as-is	SAM2-driven raw-data expansion + 3-tier threshold + 5× oversampling
Final accuracy	Count/Order/Time F1 100%	Count/Order/Time F1 100%
RTF	0.763	0.6856 (faster)

Everything we tried and rejected — with the measured reason why

TRACK	TECHNIQUE	MEASURED RESULT	REASON FOR REJECTION
YOLOv7	Self fine-tuning (Cut&Paste)	mAP 0.9904 → 0.9840, dropped	Augmented distribution diverged from the test footage
YOLOv7	SeqNMS	Suppressed real events as well	Filters sparse-but-real signals along with the noise
YOLOv7	EMA smoothing	Suppressed events	No reset — stale values persist
YOLOv7	Ghost Detector	Not applicable	Requires retraining the source paper's model
YOLOv7	ByteTrack	count 93.6% / order 91.7% / time 89.9%	Low-confidence retention blocks new tracks or over-fires
YOLOv7	WBF ensemble	False-positive explosion	Confidence-calibration mismatch between models
YOLOv7	BEV 3D fusion	Not applicable	Camera calibration data was never provided
YOLOv7	YOLO11 retraining	Order F1 −23.2 pp	Raw-data gap: 200k vs 18k images
RF-DETR	YOLOv7 code ported as-is	Order F1 stuck at 82.6–85.6%	Logic didn't match RF-DETR's confidence properties
RF-DETR	50-epoch pass 1 (no augmentation)	9 classes started over-firing	Raw-data shortage was still unaddressed
RF-DETR	Coordinate-hardcoded <code>relabel_regions</code>	Worked, but not reproducible	Replaced by whitelist tuning once proven redundant (round 10)

Takeaway — every one of these was actually implemented and A/B-tested on the server, not just cited as an idea — including the ones that failed, and why.

Five lessons that outlasted any single model

Two independent 100%-score runs, on two unrelated architectures, kept teaching the same handful of lessons. These are the ones the team would carry into the next project.

- 1 mAP is not pipeline accuracy**
Confirmed independently three separate times — YOLO11 retraining, RF-DETR's first 50-epoch pass, and knowledge distillation all raised mAP while making the actual event detection worse. This single lesson shaped every architecture decision after the first time it happened.
- 2 More data beats a newer architecture**
Every "switch to a fancier model" attempt only started working once the data problem underneath it was fixed first — YOLO11 never got that fix and stayed broken; RF-DETR got SAM2 and reached 100%.
- 3 A model's confidence quality decides how much code you have to write around it**
YOLOv7's four-layer correction stack and RF-DETR's single Noisy-OR formula solve the exact same problem — the difference is entirely in how much the raw confidence score could be trusted.
- 4 Literature-recommended tricks still need to be tested on your own data**
ByteTrack, SeqNMS, WBF, and EMA smoothing all have real published results behind them — and every technique we surveyed failed here, for reasons specific to this dataset. Reading the paper was never a substitute for running the experiment.
- 5 A debugging method can outlive the codebase it was built for**
The candidate-time back-calculation technique invented to fix one YOLOv7 ordering bug was reused, unmodified, to fix RF-DETR bugs in a completely different pipeline months later. The tooling turned out to be more durable than either model.

CONCLUSION

Same principle, different architecture, same 100%

Self fine-tuning failed, so the team pivoted to pipeline engineering. The YOLO11 attempt hit a raw-data wall — and that lesson carried straight into RF-DETR: apply the same principle ("expand raw data directly, from the target footage itself") and reach the same 100% with a shallower, more principled pipeline.

METRIC	YOLOV7	RF-DETR + SAM2
Count F1	100%	100%
Order F1	100%	100%
Time F1	100%	100%
RTF	0.763	0.6856

Architectures reaching 100%

2/2

RTF threshold — both pass with full marks

≤ 1